

Text2VectorSQL 数据合成流程介绍

omnisql

2025 年 7 月 12 日

目录

- 1 整体流程概览
- 2 Step 1: Web 表驱动的数据库合成
- 3 Step 2: Text2VectorSQL 合成
- 4 Step 3: SQL 生成
- 5 Step 4: 问题生成
- 6 环境配置
- 7 总结

- 数据合成分为四步：
 - ① Web 表驱动的数据库合成
 - ② 复杂度感知的 SQL 生成
 - ③ 风格化自然语言问题生成
 - ④ 向量化数据库构建 (Text2VectorSQL)
- 每一步都可单独运行，支持灵活扩展

Step 1: Web 表驱动的数据库合成

- 数据来源：19,935 个高质量 Web 表 (Tablib)
- 主要流程：
 - ① 解压 web_tables.json.zip
 - ② 运行 generate_schema_synthesis_prompts.py 生成数据库合成提示词
 - ③ 运行 synthesize_schema.py 生成初始数据库 schema (需实现 llm_inference)
- 增强数据库复杂度：
 - ① 运行 generate_schema_enhancement_prompts.py
 - ② 运行 enhance_schema.py (需实现 llm_inference)

Step 2: 向量化数据库构建 (Text2VectorSQL)

- 目标：为数据库表构建向量化表示，支持向量检索
- 主要流程：
 - ① 运行 `generate_schema_embedding_prompts.py` 生成向量表合成提示词
 - ② 运行 `embedding_schema.py` 生成向量数据库 schema (需实现 `llm_inference`)
 - ③ 运行 `build_sqlite_databases.py` 构建 SQLite 数据库
 - ④ 运行 `generate_tables_json.py` 生成数据库详细信息
- 输出：支持向量检索的 SQLite 数据库

Step 3: 复杂度感知 SQL 生成

- 目标：基于合成数据库生成多样化、复杂度可控的 SQL 查询
- 主要流程：
 - ① 运行 `generate_sql_synthesis_prompts.py` 生成 SQL 合成提示词
 - ② 运行 `synthesize_sql.py` 生成 SQL (需实现 `llm_inference`)
 - ③ 运行 `post_process_sqls.py` 过滤无效/重复/非 SELECT 语句
- 输出：高质量的 synthetic SQL queries

Step 4: 风格化自然语言问题生成

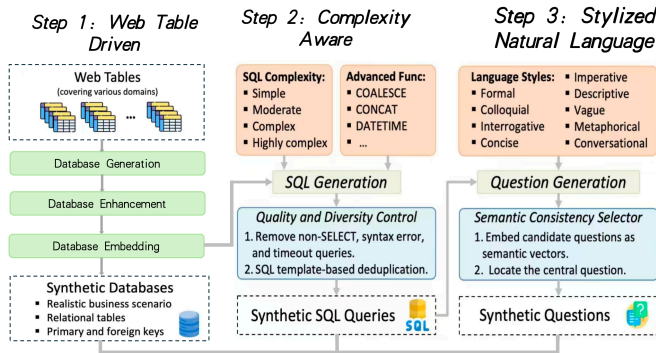
- 目标：为每条 SQL 生成多样化的自然语言问题
- 主要流程：
 - ① 运行 `generate_question_synthesis_prompts.py` 生成问题合成提示词
 - ② 运行 `synthesize_question.py` 生成问题（需实现 `llm_inference`，采样温度 0.8）
 - ③ 运行 `post_process_questions.py` 语义一致性筛选
- 输出：高质量的 `<question, SQL>` 对

- 推荐使用 Anaconda 环境
- 安装命令：

```
conda create -n omnisql_data_synthesis python=3.9.5
conda activate omnisql_data_synthesis
pip install -U sentence-transformers
pip install json-repair ijson matplotlib func_timeout
```


总结

- 本流程实现了从 Web 表到 Text2VectorSQL 的全自动数据合成
- 支持大规模、复杂、多样化的 text-to-SQL 任务研究
- 可灵活扩展、替换各环节的 LLM 或提示词



图：流程图